Mechanizmy zarządzania wątkami programowymi w zadaniu wykrywania krawędzi na obrazach cyfrowych

Zarządzanie w systemach i sieciach komputerowych – projekt

Karol Gąsior (280913), Mateusz Kunicki (272468)

Politechnika Wrocławska, W4N
Prowadzący: dr inż. Robert Wójcik, grupa G2

17.11.2026

Cel i zakres

- Wykrywanie krawędzi operatorami gradientowymi (Sobel, Prewitt, Scharr, Roberts) na obrazach do 59 Mpix.
- Pytanie badawcze: ile czasu przetwarzania dają kolejne zmiany w kodzie i w sposobie zarządzania wątkami?
- Zbadane mechanizmy:
  - optymalizacje jednowątkowe (V0–V8), w tym SIMD AVX2,
  - pula wątków z harmonogramami static / cyclic / dynamic / guided,
  - synchronizacja: bariera, redukcja maksimum, false sharing,
  - przypinanie wątków do rdzeni, OpenMP, GPU (OpenCL).
- Kryteria: czas zegarowy, przyspieszenie $S(N) = T(1)/T(N)$, efektywność $E(N) = S(N)/N$, poprawność względem wersji referencyjnej.

Sformułowanie problemu

$$G(x,y) = \sqrt{G_x^2 + G_y^2}, \qquad G_x = K_x * I, \quad G_y = K_y * I, \qquad \text{wynik} = \text{round}\left(255 \cdot \frac{G}{\max G}\right)$$

- Parametry: obraz $W \times H$, maska $k \times k$ ($k = 3, 5, 7$), liczba wątków N, harmonogram, rozmiar fragmentu.
- Złożoność: $O(WHk^2)$ dla splotu 2D, $O(WHk)$ dla maski rozdzielnej (Sobel, Prewitt, Scharr).
- Każdy piksel zależy tylko od otoczenia – zadanie łatwo zrównoleglane, ale:
  - normalizacja do max G wymaga drugiego przebiegu, redukcji i bariery,
  - wczytanie i zapis pliku są sekwencyjne (prawo Amdahla),
  - splot 3×3 wykonuje mało obliczeń na bajt – ogranicza go pamięć.

Implementacja

- C++17, CMake; biblioteka `edge_core` bez zależności.
- Jedna funkcja `runVariant()` wywołuje każdą wersję – te same granice pomiaru.
- `edge_bench` (Qt Core): eksperyment z pliku JSON, surowe wyniki do CSV, opis warunków do `meta.json`.
- Python: pomiary po kolei (blokada, kontrola zajętości CPU), wykresy i tabele do sprawozdania.

- Poprawność: każdy wariant porównywany z V0 dla obrazów od 1×1 do 1921×1081, 6 masek, 3 norm, 3 skalowań i wielu konfiguracji wątków.
- Wszystkie warianty, także GPU, dają wynik **identyczny co do bajtu** z V0.

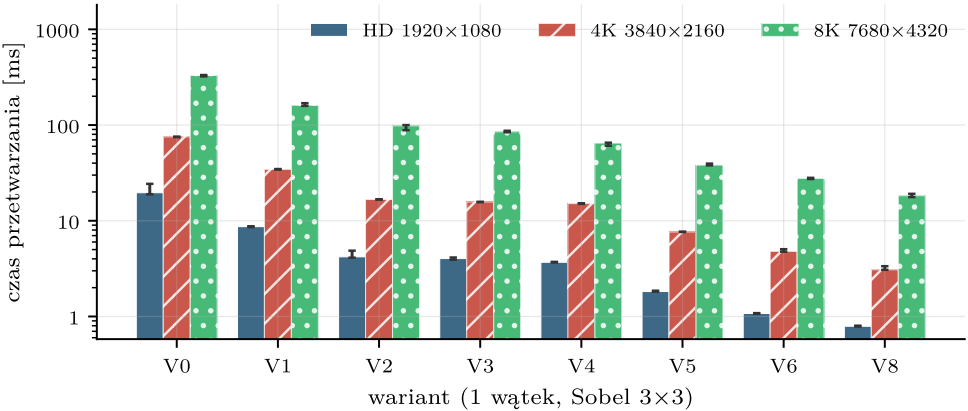
Warianty kodu – każdy dodaje jedną zmianę

Wariant	Zmiana
V0	naiwny: `std::clamp` przy każdym odczycie, maska z `std::vector`, bufory alokowane w każdym wywołaniu
V1	brzegi obrazu obsłużone poza główną pętlą
V2	maska jako stała czasu kompilacji (szablon): rozwinięte pętle, bez mnożeń przez 0
V3	splot rozdzielny: dwa sploty 1D, bufor k wierszy ($k^2 \to 2k$ mnożeń)
V4	bufory `int16`, wyrównane do 64 B, nie alokowane ponownie
V5	pętle przepisane pod automatyczną wektoryzację (AVX2)
V6	ręczne SIMD AVX2: 16 pikseli w przebiegu poziomym, 8 w pionowym
V8	V6 + przeliczenie zamiast bufora modułu ($2\times$ obliczenia, bez 8 B/piksel ruchu w pamięci)
T	pula wątków + scheduler, dowolne jądro V0–V8
OMP	`#pragma omp parallel for schedule(runtime)`, te same jądra
GPU, GPUC, GPUL	OpenCL: kernel naiwny / maska wkompilowana / kafel w pamięci lokalnej

Metodyka pomiarów

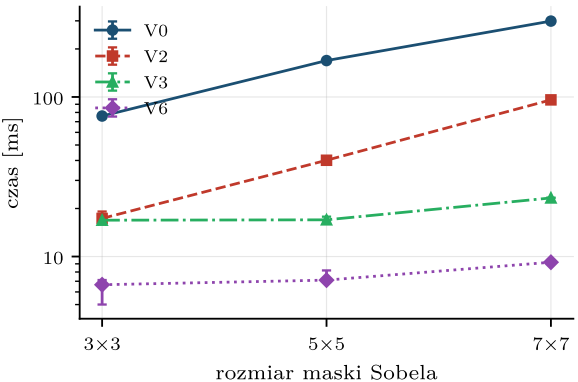
- AMD Ryzen 9 9900X 12-Core Processor: 24 CPU, 12 rdzeni, 2 grupy L3, 2 bloki rdzeni (CCD) z osobnym L3 32 MB; GeForce GTX 1050 Ti.
- Ubuntu 24.04.5 LTS, gcc 13.3.0, `-Wall -Wextra -O3 -DNDEBUG`, governor performance.
- Mierzone samo przetwarzanie (obraz w pamięci $\rightarrow$ obraz 0–255), poza E9.
- Rozgrzewka, potem 15–21 powtórzeń; wartość = mediana, słupki = min–maks.
- Konfiguracje w losowej, ale odtwarzalnej kolejności; przerwa 5 s między eksperymentami; start tylko przy zajętości CPU $< 5\,\%$.
- Obrazy: zdjęcie NASA Blue Marble przeskalowane do 0,04–59 Mpix, Kodak, Lena, obrazy syntetyczne.

Optymalizacje jednowątkowe (E1)



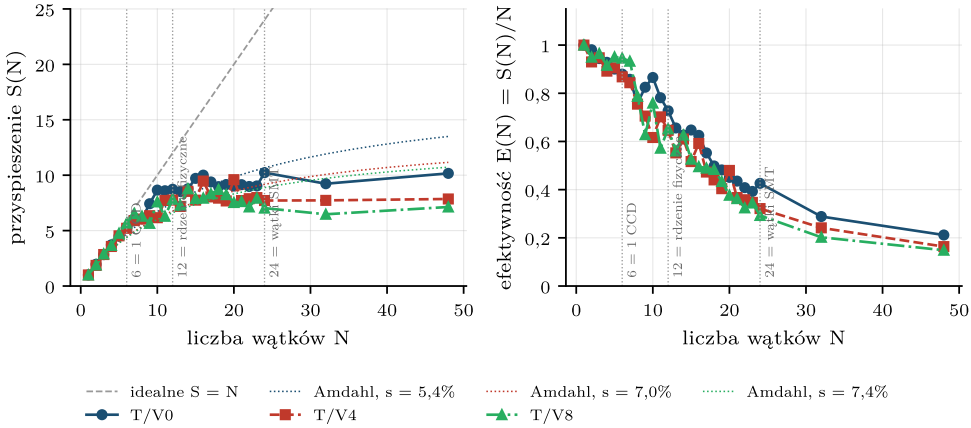
4K: V0 75,3 ms → V8 3,08 ms, przyspieszenie 24,5× na jednym wątku. Największe kroki: brzegi poza pętlą (2,19×), stała maska (2,06×), SIMD (V5, V6).

Rozmiar maski i splot rozdzielny (E2)



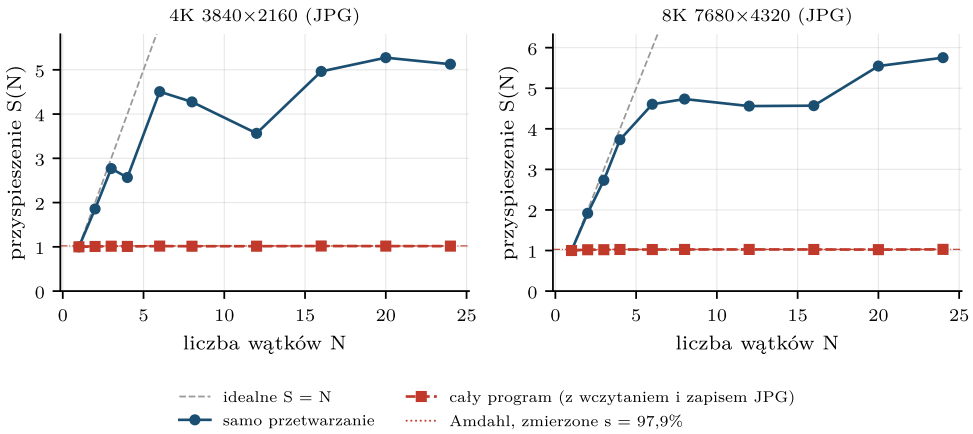
- Splot 2D rośnie z k^2: V2 17,3 ms (3×3) $\rightarrow$ 95,9 ms (7×7).
- Rozdzielny rośnie z k: V3 16,9 $\rightarrow$ 23,3 ms.
- Dla 3×3 rozdzielność prawie nic nie daje, dla 7×7 – $4\times$.
- V0$\rightarrow$V6: 11,4$\times$ (3×3), 32,4$\times$ (7×7).

Liczba wątków (E3), obraz 4K



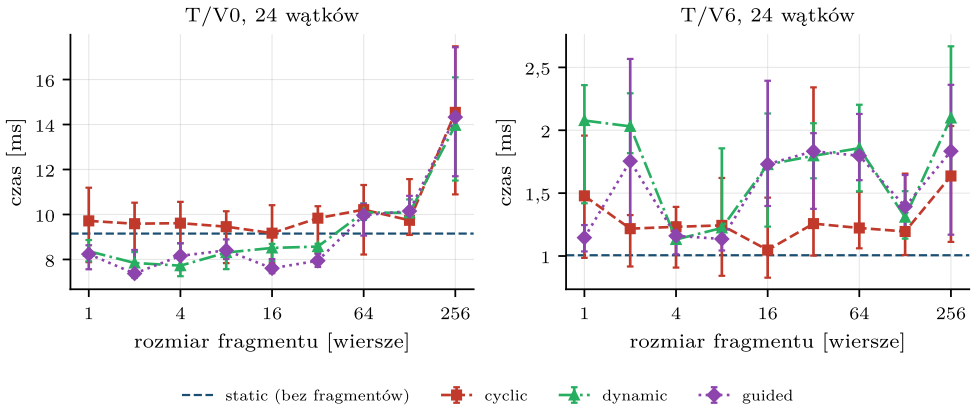
Maks. S: T/V0 10,2 (N = 24,0), T/V8 8,77 (N = 14,0). Szybsze jądro → większy udział części sekwencyjnej i pamięci → mniejsze S.

Prawo Amdahla w praktyce (E9)



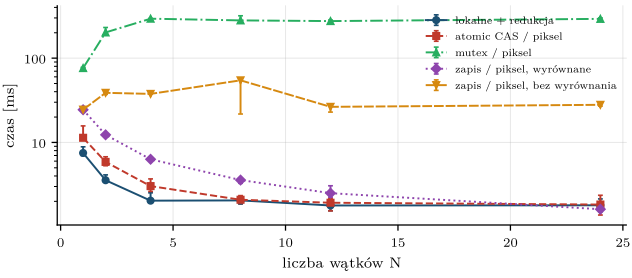
4K: wczytanie JPG 37,2 ms, zapis 138 ms, przetwarzanie 3,69 ms → 0,719 ms. Część sekwencyjna $s = 97,9\,\%$, więc cały program przyspiesza tylko 1,02×.

Harmonogramy i rozmiar fragmentu (E4), 24 wątki



T/V6: najlepiej static (1,01 ms), najgorzej dynamic, 256 (2,10 ms). Praca na wiersz jest stała, więc równoważenie dynamiczne nie ma czego wyrównywać, a każdy fragment kosztuje operację atomową i ponowne wypełnienie bufora k wierszy.

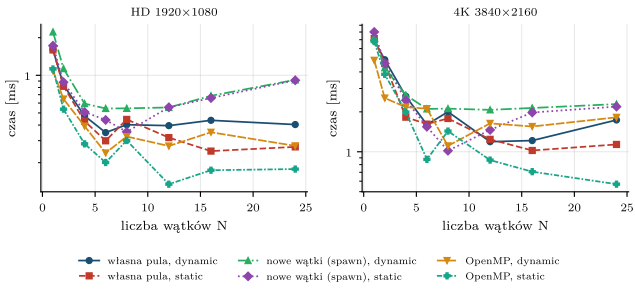
Synchronizacja: wyznaczanie maksimum (E8b)



24 wątki, 4K:

- lokalne maksimum + redukcja: 1,80 ms,
- atomic CAS: 1,83 ms (zapisy rzadkie),
- zapis na piksel, osobne linie cache: 1,61 ms,
- to samo w jednej linii (false sharing): 27,9 ms,
- mutex na piksel: 292 ms.

Mechanizm wątków i topologia (E6, E10)

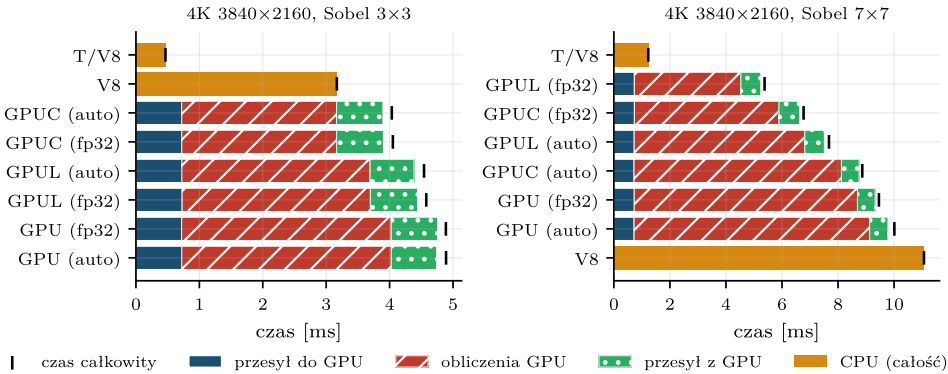


4K, 24 wątki, static:

- OpenMP 0,570 ms,
- własna pula 1,14 ms,
- nowe wątki w każdym etapie 2,19 ms.

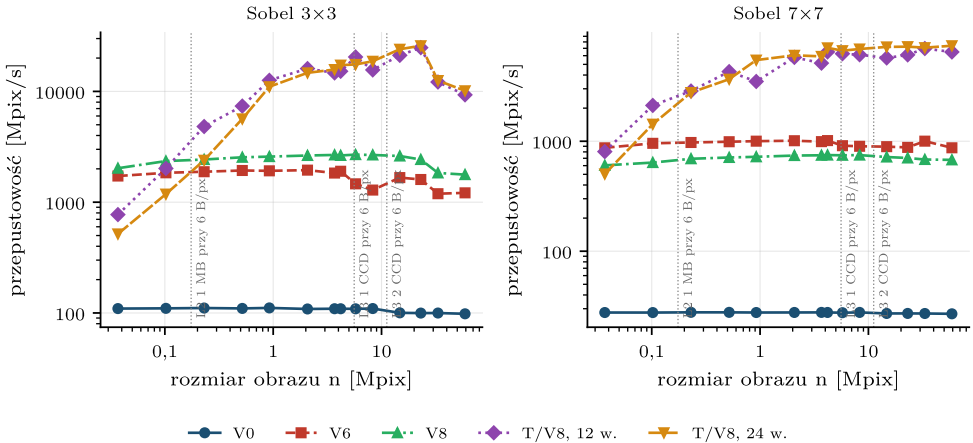
Przypinanie (8K, 12 wątków, T/V8): 1 wątek na rdzeń 1,63 ms, bez przypinania 2,97 ms.

GPU (E5): GeForce GTX 1050 Ti, OpenCL



4K, Sobel 3×3: GPUC 4,04 ms, w tym przesył 0,722 + 0,731 ms; CPU T/V8 0,466 ms. Na tym sprzęcie GPU nie wygrywa w żadnym rozmiarze.

Rozmiar danych wejściowych (E13)



Wątki opłacają się od ok. 0,23 Mpix (12 w.) i 0,52 Mpix (24 w.). Przepustowość jednego wątku spada, gdy dane przestają mieścić się w L3.

Wynik filtru

obraz wejściowy (skala szarości)



Sobel 3×3, norma L_2



Sobel 7×7, norma L_2



$|L_1 - L_2| \times 8$



Wnioski

1. Optymalizacje jednowątkowe dały 24,5× (4K) – więcej niż 24 wątki dla kodu naiwnego (10,2×).
2. Im szybsze jądro, tym gorsze skalowanie: ograniczeniem staje się pamięć i część sekwencyjna (T/V8: maks. 8,77×).
3. Wczytanie i zapis JPG ($s = 97,9\,\%$) przekreślają zysk z wątków dla całego programu.
4. Sposób synchronizacji zmienia czas o rzędy wielkości: false sharing 27,9 ms, mutex 292 ms wobec 1,80 ms.
5. Przy stałym koszcie wiersza najlepszy jest prosty podział statyczny.
6. GPU klasy GTX 1050 Ti przegrywa z 12-rdzeniowym CPU z AVX2 – przesył i słaba moc obliczeniowa.

Do dokończenia: wariant zaproponowany przez AI (porównanie z V1–V8).